4 Gestione della configurazione

OBIETTIVI FORMATIVI

- Comprendere i principali compiti e responsabilità della gestione della configurazione del software
- Essere a conoscenza dei contenuti di un piano di gestione della configurazione
- Apprezzare l'interazione tra il ruolo della gestione della configurazione nello sviluppo del software e le capacità degli strumenti di supporto

Sono necessarie procedure attente per gestire il vasto numero di elementi (componenti del codice sorgente, documentazione, richieste di modifica, ecc.) che vengono creati e aggiornati nel corso della vita di un grande sistema software. Ciò è particolarmente vero nei progetti di sviluppo distribuito. Si chiama **gestione della configurazione**.

Nel corso di un progetto di sviluppo software vengono prodotti non pochi documenti. Anche questi documenti vengono modificati di volta in volta. Gli errori devono essere corretti, le richieste di modifica devono essere gestite, ecc. Pertanto, in ogni momento durante un progetto, possono esistere versioni diverse dello stesso documento in parallelo.

Spesso anche un sistema software in sé non è monolitico. I sistemi software esistono in diverse versioni o configurazioni. Versioni diverse si verificano quando le modifiche vengono implementate dopo che il sistema è stato consegnato al cliente. Di tanto in tanto, il cliente deve quindi confrontarsi con una nuova versione. Durante lo sviluppo possono esistere anche versioni differenti di componenti di un sistema. Ad esempio, se una richiesta di modifica è stata approvata, un programmatore può implementare tale modifica riscrivendo uno o più componenti. Un altro programmatore, tuttavia, potrebbe ancora utilizzare la versione precedente degli stessi componenti.

Diverse configurazioni si verificano anche se un insieme di componenti può essere assemblato in un sistema in più di un modo. L'Amsterdam Compiler Kit (ACK) consiste in un insieme di programmi per sviluppare compilatori per linguaggi simili ad ALGOL (Tanenbaum *et al.*, 1983). I componenti importanti dell'ACK sono:

- front-end per linguaggi come Pascal, C o Modula-2 (un front-end per una lingua traduce i programmi in quella lingua nel codice intermedio universale, EM);
- diversi ottimizzatori EM;
- back-end, che traducono il codice EM in codice assembler per una varietà di macchine reali.

Si ottiene quindi un compilatore selezionando un front end per un linguaggio specifico, un back end per una macchina specifica e, facoltativamente, uno o più ottimizzatori. Ogni compilatore è una configurazione, una certa combinazione di elementi del sistema ACK. Il sistema ACK è un esempio di una linea di prodotti, di un'era precedente all'utilizzo di tale nozione.

I compiti chiave della gestione della configurazione sono discussi nella Sezione 4.1 . Un piano di gestione della configurazione stabilisce le procedure che descrivono come affrontare la gestione della configurazione. I contenuti di questo documento sono discussi nella Sezione 4.2 . La gestione della configurazione è spesso supportata da strumenti. La discussione di questi strumenti è in gran parte rinviata al Capitolo 15 .

4.1 COMPITI E RESPONSABILITÀ

La gestione della configurazione riguarda la gestione di tutti gli artefatti prodotti nel corso di un progetto di sviluppo software. Sebbene la gestione della configurazione svolga un ruolo anche durante la fase operativa di un sistema, quando diverse combinazioni di componenti possono essere assemblate in un sistema e vengono generate nuove versioni di un sistema, la discussione di seguito è incentrata sul ruolo della gestione della configurazione durante lo sviluppo del sistema.

Assumeremo per il momento che, in qualsiasi momento, esiste una versione ufficiale della serie completa di documenti relativi al progetto. Questa è chiamata linea di **base (baseline)**. Una linea di base è "una specifica o un prodotto che è stato formalmente rivisto e concordato, che in seguito funge da base per un ulteriore sviluppo e che può essere modificato solo attraverso procedure formali di controllo delle modifiche" (IEEE610, 1990). Pertanto, la baseline è il database del progetto condiviso, contenente tutti gli elementi approvati. La baseline può essere o meno archiviata in un database reale e supportata da strumenti per aiutare a recuperare e aggiornare i suoi elementi. Gli elementi contenuti nella baseline sono gli **elementi di configurazione**. Un elemento di configurazione è "un'aggregazione di hardware, software o entrambi, designata per la gestione della configurazione e trattata come una singola entità nel processo di gestione della configurazione" (IEEE610, 1990). Possibili elementi di configurazione sono:

- componenti del codice sorgente,
- la specifica dei requisiti,
- la documentazione progettuale,
- il piano di prova,
- casi test,
- risultati del test,
- il manuale utente.

Ad un certo punto, la baseline conterrà una specifica dei requisiti. Col passare del tempo, verranno aggiunti elementi: documenti di progettazione, componenti del codice sorgente, rapporti di test, ecc. Uno dei compiti principali della gestione della configurazione è mantenere l'integrità di questo insieme di artefatti.

Ciò è particolarmente importante se devono essere incorporate modifiche. Supponiamo che, durante il test, venga scoperto un grave difetto in qualche componente. Dobbiamo quindi tornare sui nostri passi e correggere non solo quel componente, ma anche i relativi documenti

di progettazione ed eventualmente anche la specifica dei requisiti. Ciò potrebbe influire sul lavoro svolto da altre persone che utilizzano ancora la vecchia versione. Peggio ancora, qualcun altro potrebbe voler apportare modifiche allo stesso componente contemporaneamente. La gestione della configurazione si occupa di controllare il rilascio e la modifica di questi elementi durante tutto il ciclo di vita del software.

Il modo per farlo è avere una libreria o un database condivisi che contenga tutti gli elementi approvati, la "baseline". L'aggiunta di un articolo a questo database o la modifica di un articolo è soggetto a uno schema di approvazione formale. Per i progetti più grandi, questa è responsabilità di un organismo separato, il gruppo di controllo di configurazione (o modifica) (CCB – Configuration Change Board). La BCC garantisce che qualsiasi modifica alla baseline sia adeguatamente autorizzata ed eseguita. Il CCB è composto da personale delle varie parti coinvolte nel progetto, come lo sviluppo, i test e l'assicurazione della qualità.

Qualsiasi modifica proposta alla baseline è chiamata richiesta di modifica (change request CR). Una richiesta di modifica può riguardare un errore riscontrato in qualche componente, una discrepanza rilevata tra un documento di progettazione e la sua implementazione, un miglioramento causato da requisiti utente modificati, ecc. Una richiesta di modifica viene gestita come segue (vedere anche la Figura 4.1):

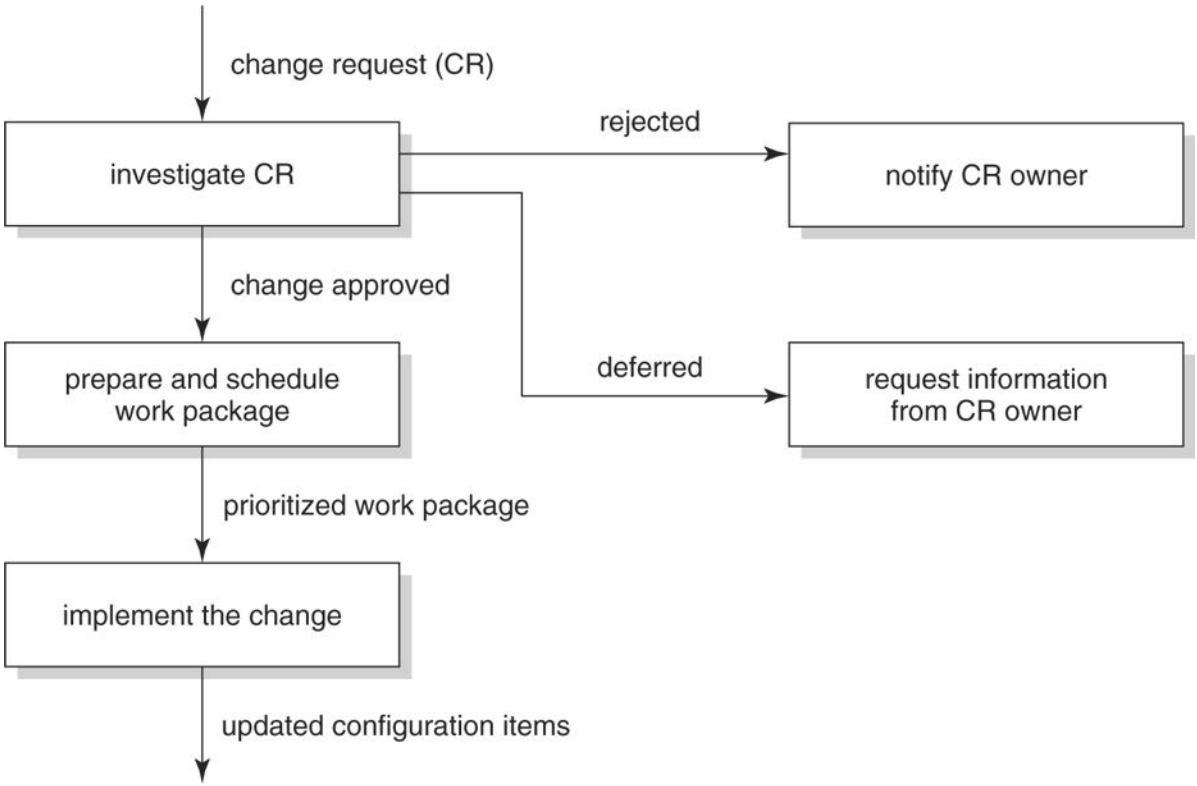


Figura 4.1 Flusso di lavoro di una richiesta di modifica

- La modifica proposta viene presentata alla CCB. Per poter valutare la modifica proposta, la BCC necessita di informazioni su come la modifica influisca sia sul prodotto che sul

processo di sviluppo. Ciò include informazioni sulla quantità stimata di codice nuovo o modificato, requisiti di test aggiuntivi, relazione con altre modifiche, costi potenziali, complessità della modifica, gravità del difetto (se lo riguarda), risorse necessarie, ecc. Di solito, viene fornito un apposito modulo di richiesta di modifica per specificare le informazioni necessarie alla BCC.

- La CCB valuta la richiesta di modifica. La richiesta di modifica può essere approvata, rifiutata o differita se sono necessarie ulteriori informazioni. Se la richiesta viene approvata, alla fine si ottiene un pacchetto di lavoro che deve essere pianificato.
- Il CCB si assicura che tutti gli elementi di configurazione interessati vengano eventualmente aggiornati di conseguenza. La gestione della configurazione fornisce un modo per stabilire lo stato di tutti gli elementi e, di conseguenza, dell'intero progetto.

Pertanto, la gestione della configurazione non consiste solo nel tenere traccia di tutte le diverse versioni degli elementi di un sistema; comprende anche attività di gestione del flusso di lavoro. Il processo illustrato nel diagramma di transizione dello stato in Figura 4.1 , ad esempio, descrive cosa accade nel ciclo di vita di una richiesta di modifica. Il modello di processo così definito esemplifica come è possibile gestire il flusso di lavoro delle richieste di modifica.

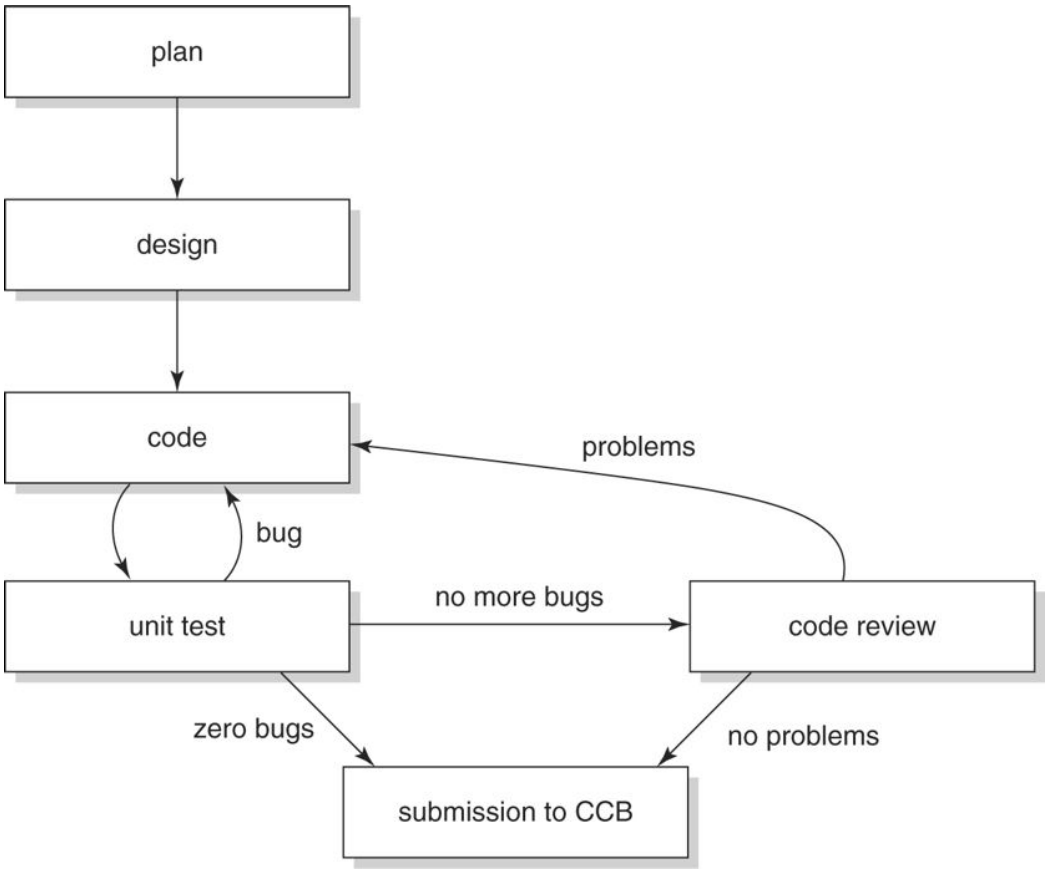


Figura 4.2 Diagramma di transizione dello stato delle attività di sviluppo

Allo stesso modo, il diagramma di transizione dello stato nella Figura 4.2 mostra il flusso di lavoro delle attività degli sviluppatori durante lo sviluppo di un componente di sistema. Mostra i possibili stati di un componente del sistema e le transizioni tra di essi. Ad esempio, dopo che un componente è stato codificato, viene testato per unità. Se vengono rilevati bug durante il test delle unità, è necessaria un'ulteriore codifica. In caso contrario, il componente entra nella fase di revisione. Se la revisione rivela problemi, si rientra nella fase di codifica. In caso contrario, il componente viene sottoposto alla BCC per l'approvazione formale. Infine, se il test unitario non rivela errori, la fase di revisione viene saltata.

Se i componenti vengono mantenuti sotto il controllo della configurazione, la gestione della configurazione può essere utilizzata anche per gestire il flusso di lavoro delle attività di sviluppo. Le modifiche allo stato di un componente attivano quindi le attività successive, come indicato nel modello del flusso di lavoro di sviluppo.

Dobbiamo fare in modo che gli schemi del flusso di lavoro non riducano inutilmente il lavoro quotidiano delle persone coinvolte nel progetto. I nuovi elementi non devono essere aggiunti alla baseline fino a quando non sono stati accuratamente esaminati e testati. Gli elementi del database condiviso possono essere utilizzati liberamente dai partecipanti. Se un articolo deve essere modificato, la persona responsabile dell'implementazione della modifica riceve una copia di quell'articolo e l'articolo viene temporaneamente bloccato, in modo che altri non possano aggiornare contemporaneamente l'articolo. La persona che implementa la modifica è libera di armeggiare con la copia. Dopo che la modifica è stata accuratamente testata, viene rinviata alla CCB. Una volta che la BCC lo ha approvato, l'articolo rivisto viene incluso nel database, la modifica stessa viene documentata con l'articolo e l'articolo viene sbloccato di nuovo. Una sequenza di modifiche documentate fornisce quindi una cronologia delle revisioni di quell'elemento.

Quando un elemento viene modificato, viene conservata anche la vecchia versione. La vecchia versione deve ancora essere utilizzata da altri finché non si sono adattati al cambiamento. Inoltre, potremmo voler tornare alla vecchia versione se viene richiesta un'altra modifica. Abbiamo quindi versioni diverse di uno stesso articolo e dobbiamo essere in grado di distinguerle. Questo può essere fatto attraverso uno schema di numerazione, in cui ogni nuova versione è identificata dal numero successivo più alto. Otteniamo quindi, per un componente X, le versioni X.0, X.1, X.2 e così via.

In un ambiente più sofisticato, potremmo persino creare diversi rami di revisioni. La Figura 4.3 fornisce un esempio di tale sviluppo biforcuto. Nell'esempio, il componente X.2.1 può essere il risultato della correzione di un bug nel componente X.2. Il componente X.3 può riguardare un miglioramento di X.2. Va notato che unire nuovamente quei percorsi di sviluppo paralleli può essere difficile. Inoltre, gli schemi di numerazione tendono presto a diventare incomprensibili.

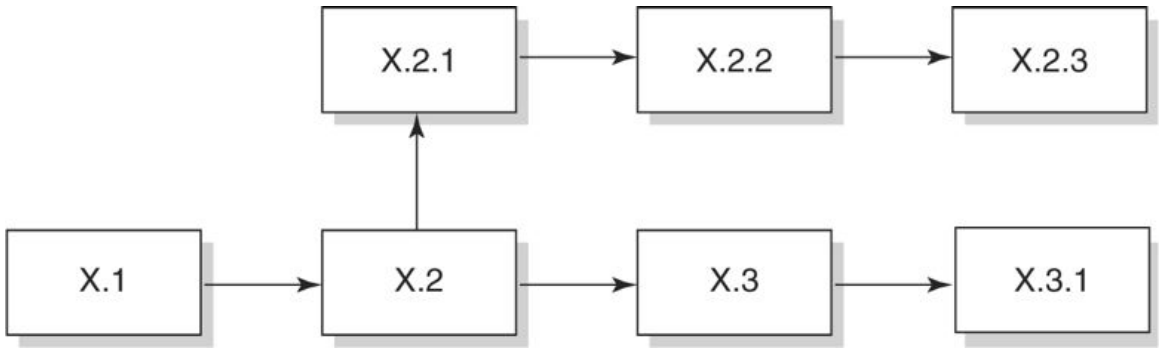


Figura 4.3Percorsi di sviluppo paralleli

La gestione della configurazione è generalmente supportata da potenti strumenti. Dart (**1990**) classifica le funzionalità degli strumenti di gestione della configurazione del software (SCM) come segue:

- **Componenti** Gli strumenti SCM supportano l'archiviazione, il recupero e l'accesso ai componenti. È possibile memorizzare diverse versioni di un componente, stabilire linee di base e creare rami di revisioni.
- **Struttura** Gli strumenti SCM supportano la rappresentazione e l'utilizzo della struttura di un sistema costituito dai componenti e dalle relative interfacce. In termini di punti di vista dell'architettura, questo è il punto di vista dell'implementazione (vedi **Sezione 11.3**).
- **Costruzione** Gli strumenti SCM supportano la costruzione di una versione eseguibile del sistema. Per impostazione predefinita, viene utilizzata l'ultima versione di tutti gli elementi di configurazione, ma è anche possibile rigenerare versioni precedenti del sistema.
- **Auditing/Controllo** SCM consentono di seguire le tracce: quali modifiche sono state apportate a questo componente, chi ha apportato tali modifiche e perché. In questo modo viene mantenuto un archivio ricercabile del sistema.
- **Contabilità** L'archivio ricercabile permette di raccogliere statistiche sul sistema e sul processo di sviluppo. Potremmo, ad esempio, cercare componenti che sono cambiati spesso e ipotizzare la qualità di tali componenti.
- **Tracciabilità** SCM possono essere utilizzati a fini di tracciabilità. Se vengono memorizzate informazioni sufficienti, potremmo ricondurre i difetti ai requisiti, analizzare l'impatto delle modifiche e così via.
- **Gestione del processo** possono supportare gli utenti nella selezione delle attività e nell'esecuzione di tali attività nel contesto appropriato. Ad esempio, lo strumento può aiutare ad assegnare la gestione di una richiesta di modifica a un determinato sviluppatore e fornire automaticamente allo sviluppatore uno spazio di lavoro contenente i componenti che devono essere modificati.
- **Team** SCM possono supportare la collaborazione, ad esempio generando uno spazio di lavoro per un gruppo di sviluppatori che collaborano, notando conflitti tra gli sviluppatori e così via.

Molti strumenti SCM utilizzano il modello di configurazione **orientato alla versione.** Una modifica fisica in un componente si traduce quindi in una nuova versione e versioni diverse sono quindi caratterizzate dalla loro differenza. Alcuni strumenti utilizzano le modifiche logiche, anziché fisiche, come unità di lavoro di base nella gestione della configurazione. Questo modello **orientato al cambiamento** offre un modo più intuitivo di lavorare e può prevenire molti errori dell'utente durante la configurazione di un sistema. Invece di identificare una configurazione da una sequenza arcana di numeri, ora è identificata da una baseline più una serie di modifiche. L'insieme delle modifiche potrebbe essere vuoto. Specifichiamo quindi

- baseline X più "risolvere il problema delle dimensioni della tabella"

piuttosto che

- {X.3.1, Y.2.7, Z.1.4, …}.

Come notato, gli strumenti SCM offrono anche aiuto nella costruzione ("costruzione") di una versione eseguibile del sistema. Si scrive un 'programma' che identifica i vari componenti del sistema richiesto e le loro reciproche dipendenze. Il sistema in questione viene quindi generato eseguendo questo 'programma': i componenti vengono recuperati automaticamente dal database contenente i componenti del codice sorgente e tutti i componenti vengono tradotti e collegati tra loro in un sistema eseguibile. Se il sistema è abbastanza intelligente, solo i componenti che sono stati modificati vengono tradotti di nuovo.

I primi strumenti SCM enfatizzavano le attività orientate al prodotto della gestione della configurazione. Fornivano funzionalità per bloccare e sbloccare elementi, per numerare automaticamente le revisioni e, per impostazione predefinita, per fornire agli utenti l'ultima versione di un elemento. Se un elemento è stato modificato, hanno chiesto all'utente di documentare la modifica. Gli odierni strumenti SCM forniscono sempre più anche le altre funzionalità e sono diventati un ingrediente chiave per la gestione dello sviluppo di software moderno, distribuito e globale. Gli strumenti per la configurazione e la gestione delle versioni sono discussi più ampiamente nel **Capitolo 15** .

Da un lato, la gestione della configurazione prevede procedure su come gestire le modifiche e le versioni dei documenti. D'altra parte, consiste in uno strumento di supporto per mantenere la cronologia delle versioni e garantire una versione aggiornata del sistema. Nello sviluppo guidato dalla pianificazione, entrambi gli aspetti sono importanti. Nei progetti agili, l'enfasi è sul supporto degli strumenti. I progetti agili favoriscono l'integrazione continua, in cui il lavoro individuale di uno sviluppatore viene rapidamente integrato con altre parti del sistema e quindi vengono eseguiti test per assicurarsi che tutto funzioni ancora come previsto. Un tale ciclo di sviluppo può richiedere alcune ore o, al massimo, un giorno. Alla fine della giornata, si ha di nuovo un sistema in esecuzione che supera tutti i test. Questo processo è noto come *build giornaliera.*

4.2 PIANO DI GESTIONE DELLA CONFIGURAZIONE

Le procedure per la gestione della configurazione sono stabilite in un documento, il piano di gestione della configurazione. Per i contenuti di questo piano, seguiamo lo standard appropriato, IEEE828 (1998). Questo documento descrive i metodi per identificare gli elementi di configurazione, per controllare le richieste di modifica e per documentare l'implementazione di tali richieste di modifica. Un sommario di esempio per un piano di gestione della configurazione è riportato nella Tabella 4.1 .

La sezione Gestione SCM descrive come viene organizzato il progetto. Particolare attenzione è riservata alle responsabilità che incidono direttamente sulla gestione della configurazione: come vengono gestite le richieste di modifica, come vengono chiuse le fasi di sviluppo, come viene mantenuto lo stato del sistema, come vengono identificate le interfacce tra i componenti? Viene inoltre delineata la relazione con altre organizzazioni funzionali, come lo sviluppo del software e l'assicurazione della qualità.

La sezione Attività SCM descrive come verrà identificata e controllata una configurazione e come verrà contabilizzato e segnalato il suo stato. Una configurazione è identificata da una baseline: una descrizione dei componenti di tale configurazione. Tale configurazione deve essere formalmente approvata dalle parti coinvolte.

Tabella 4.1 Contenuto di un piano di gestione della configurazione software

Parte	Descrizione
1. Introduzione	Scopo, ambito, termini e riferimenti principali
2. Gestione SCM	Organizzazione, responsabilità e autorità
3. Attività SCM	Identificazione degli elementi di configurazione, controllo, contabilità dello stato e processo di auditing
4. Orari SCM	Coordinamento con altre attività
5. Risorse SCM	Strumenti, risorse umane e informatiche
6. Manutenzione del piano SCM	Come mantenere aggiornato il piano

Se si vuole controllare un progetto di sviluppo software, sono necessarie procedure chiare e precise rispetto all'elaborazione delle richieste di modifica. Un comitato di controllo della configurazione (Configuration Control Board - CCB) di solito ha la responsabilità di valutare e approvare o rifiutare le modifiche proposte. L'autorità, la responsabilità e l'appartenenza alla CCB devono essere dichiarate. Poiché i componenti software sono generalmente incorporati in una libreria, è necessario stabilire anche le procedure per il controllo di questa libreria.

Per poter controllare un progetto di sviluppo software, i dati devono essere raccolti ed elaborati. Le informazioni normalmente richieste includono: lo stato attuale dei componenti, delle versioni e delle richieste di modifica, nonché i rapporti sulle modifiche approvate e la loro implementazione.

Le modifiche agli elementi di configurazione possono influire sugli elementi al di fuori dell'ambito del piano, come gli elementi hardware. Questi elementi esterni devono essere identificati e le loro interfacce controllate. Allo stesso modo, devono essere identificate e controllate le interfacce con gli elementi sviluppati al di fuori del progetto.

4.3 USO DI GIT e GITHUB

Un sistema di controllo delle versioni (VCS) è un programma o un set di programmi che tiene traccia delle modifiche a una raccolta di file. Uno degli obiettivi di un VCS è richiamare facilmente le versioni precedenti di singoli file o dell'intero progetto. Un altro obiettivo è consentire a più membri del team di lavorare a un progetto, anche sugli stessi file, contemporaneamente senza influenzare il lavoro degli altri. Un altro nome per un VCS è un sistema di gestione della configurazione software (SCM). I due termini sono spesso usati in modo intercambiabile.

Git è un SCM *distribuito*, il che significa che la cronologia completa di un progetto è archiviata sia sul client *che* sul server. Puoi modificare i file senza una connessione di rete, archiviarli localmente e sincronizzarli con il server quando una connessione diventa disponibile. Se un server va giù, hai comunque una copia locale del progetto. Tecnicamente, non hai nemmeno bisogno di un server.

Per comprendere Git, devi comprendere la terminologia. Ecco un breve elenco di termini che gli utenti Git usano di frequente.

Il concetto fondamentale di git, è quello di **repository**. Un repository contiene tutti i file del tuo progetto e la cronologia delle revisioni di ogni file. È una delle parti essenziali che ti aiuta a collaborare con le persone. Puoi usare i repository per gestire il tuo lavoro, tracciare le modifiche, archiviare la cronologia delle revisioni e lavorare con altri.

Commit: quando usato come verbo, commit significa creare un oggetto commit. Questa azione prende il nome da commit in un database. Significa che stai eseguendo il commit delle modifiche che hai apportato in modo che anche altri possano vederle.

Branch: un branch è una serie denominata di commit collegati. Il commit più recente su un branch è chiamato head. Il branch predefinito, che viene creato quando si inizializza un repository, è chiamato main, spesso denominato master in Git. La head del branch corrente è denominata HEAD. I branch sono una funzionalità incredibilmente utile di Git perché consentono agli sviluppatori di lavorare in modo indipendente (o insieme) nei branch e in seguito unire le loro modifiche nel branch predefinito.

Comandi e opzioni: le operazioni Git vengono eseguite tramite comandi come git push e git pull. I comandi sono spesso accompagnati da opzioni, che utilizzano trattini (-) o doppi trattini (--). Ad esempio, git reset --hard.

Github è un sito che permette di creare e memorizzare in modo semplice repository di git.

Come creare un repository su github

Puoi creare un nuovo repository sul tuo account personale o su qualsiasi organizzazione per la quale disponi di autorizzazioni sufficienti. Su github lo puoi fare direttamente dal sito.

Ogni repository ha una **visibilità.** Gli archivi pubblici sono accessibili a chiunque su Internet. I repository privati sono accessibili solo a te, alle persone con cui condividi esplicitamente l'accesso e, per i repository dell'organizzazione, ad alcuni membri dell'organizzazione.

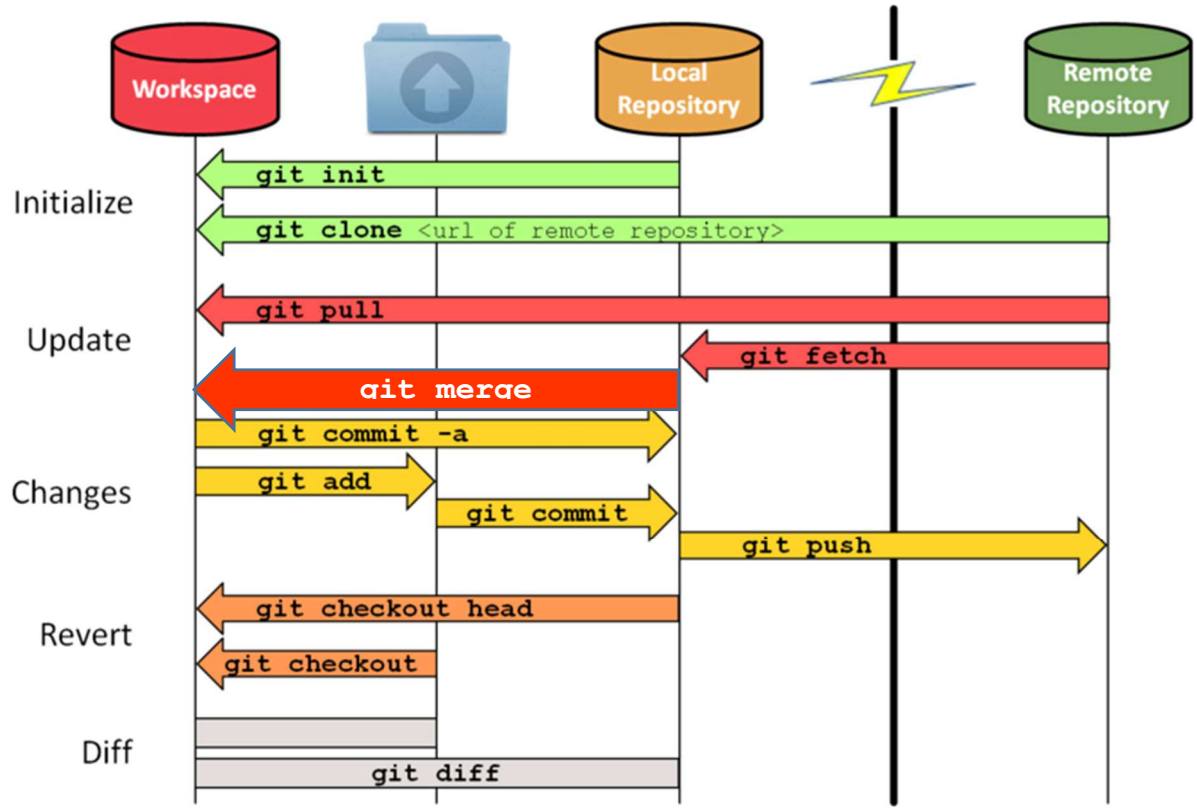


Figure 4.4 Comandi git guarda anche https://www.baeldung.com/ops/git-guide

Inizializzazione - git init

Converte un progetto (area di lavoro) esistente senza controllo delle versioni in un repository git o crea un nuovo repository git vuoto. L'esecuzione di questo comando crea una sottodirectory.git, che contiene tutti i metadati per il nuovo repository. Questi metadati includono sottodirectory per oggetti, ref e file template. HEAD

Clonare un repo esistente – git clone

Scarica un repository git esistente sul tuo computer locale – come opzione passa l'url del repo remoto. Differenza tra git init vs git clone: ad alto livello, possono essere entrambi utilizzati per inizializzare un nuovo repository git. Tuttavia, git clone copia i dati da un repository esistente.

Aggiunta di file – git add

Aggiunge modifiche nell'area di lavoro all'area di staging. Area di lavoro → Area di staging

Eseguire il commit – git commit

Aggiunge le modifiche nell'area di staging al repository Git locale: Area di staging → Repository git locale. Si può fare l'add e il commit contemporanemante con git commit -a. Il commit richiede un messaggio che viene poi memorizzato.

git push e git pull e git fetch

Push: aggiunge le modifiche nel repository git locale al repository remoto

Pull: aggiorna il repository git locale e il workspace dal repository git remoto corrispondente

Fetch: aggiorna solo il repository git local (ma non il workspace)

unione di git git merge

Integrare le modifiche locali con quelle remote in una sola. Si può fare tipicamente dopo un fetch. Nel caso in cui ci siano conflitti essi verranno segnalati e vanni risolti prima di poter fare il commit. Tipicamente bisogna aprire il file e vedere dove sono le differenze tra la copia locale e quella remota e correggere a mano queste differenze.

4.4 Sistemi di controllo centralizzati vs distribuiti

la principale differenza tra il controllo della versione centralizzato e quello distribuito (in inglese Centralized and Distributed Version Control) è che, nel controllo della versione centralizzato, le versioni vengono salvate nel repository remoto, mentre nel controllo della versione distribuito, le versioni possono essere salvate nel repository remoto e nei repository locali delle macchine locali.

Controllo della versione centralizzato

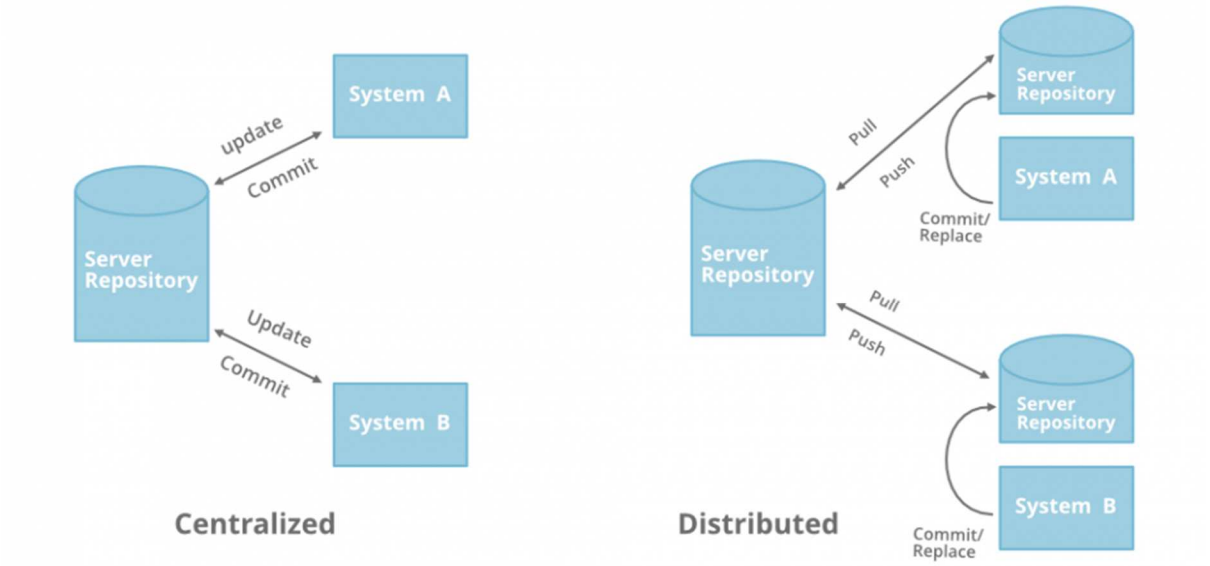
Nel controllo delle versioni centralizzato, c'è un insieme di computer locali e un singolo server. Il server funziona come posizione centrale. Quando uno sviluppatore modifica il codice sorgente nella sua macchina locale, tali modifiche vengono salvate nel server centrale. La copia di lavoro è disponibile nella macchina locale, ma le versioni vengono salvate nel server. In altre parole, la posizione centrale gestisce le versioni.

Un repository è un'area di archiviazione che memorizza i codici sorgente. Chiamiamo il repository del server centrale il repository remoto. Se si verifica un errore nella posizione centrale (server), lo sviluppatore non può accedere alle versioni. Inoltre, lo sviluppatore non sarà in grado di accedere al repository nella posizione centrale a causa di problemi di connettività di rete. Questi sono alcuni svantaggi di un sistema di controllo delle versioni centralizzato.

Controllo della versione distribuita

Il controllo delle versioni distribuito fornisce soluzioni ai limiti del controllo delle versioni centralizzato. C'è una serie di computer e un server. Il server dispone di un repository remoto. Allo stesso tempo, ogni computer ha il proprio repository locale. Non ci sono problemi di guasto del server poiché ogni macchina ha un repository locale. Il repository è distribuito su più macchine. Pertanto, lo chiamiamo un sistema di controllo della versione distribuito.

Inoltre, in questa configurazione, ogni sviluppatore ottiene il proprio repository. Può apportare una modifica nel repository locale e condividerlo con gli altri sviluppatori. Inoltre, non ci sono interferenze da altri archivi. Ogni repository tiene traccia della cronologia, esegue il push e il pull delle modifiche ad altri repository e supporta l'unione. Inoltre, se non c'è connettività di rete, lo sviluppatore può creare versioni nel proprio repository locale. Può eseguire il push delle modifiche al repository remoto quando la connessione Internet è disponibile.



SVN (subversion) è un tipico sistema di controllo delle versioni centralizzator, mentre GIT è un tipico sistema di versioning distribuito.

4.5 RIEPILOGO

La gestione della configurazione riguarda la gestione di tutti gli artefatti prodotti nel corso di un progetto di sviluppo software. Prevede le seguenti principali attività:

- Gli elementi di configurazione devono essere identificati e definiti. Un elemento di configurazione è una raccolta di elementi trattati come un'unità ai fini della gestione della configurazione. Esempi di possibili elementi di configurazione sono la specifica dei requisiti, un componente software, un rapporto di prova e la documentazione per l'utente.
- È necessario controllare il rilascio e la modifica degli elementi di configurazione durante il ciclo di vita del software. Ciò significa che devono essere stabilite procedure ordinate su chi è autorizzato a modificare o rilasciare elementi di configurazione.
- Lo stato degli elementi di configurazione e delle richieste di modifica deve essere registrato e riportato. Ad esempio, lo stato di una richiesta di modifica può essere proposto, approvato, rifiutato o incorporato.

Per progetti più grandi, di solito viene istituita un comitato di controllo della configurazione (CCB). La CCB è responsabile della valutazione di tutte le richieste di modifica e del mantenimento dell'integrità della serie completa di documenti relativi a un progetto. I suoi compiti e le ulteriori procedure per la gestione della configurazione sono definiti in un documento separato, il piano di gestione della configurazione.

La storia e lo sviluppo della gestione della configurazione è strettamente legata alla storia e allo sviluppo degli strumenti di gestione della configurazione. All'inizio, questi strumenti enfatizzavano la registrazione delle modifiche ai file fisici. C'era poco supporto per gli aspetti del processo. Gli attuali sistemi di gestione della configurazione affrontano anche aspetti di processo (gestione del flusso di lavoro) e molti hanno adottato una visione delle configurazioni orientata al cambiamento così come o invece di una visione orientata alla versione. Sempre di più, gli strumenti di gestione della configurazione funzionano come strumenti di gestione dei documenti in quanto supportano la cooperazione tra un gruppo di persone, possibilmente distribuite su più siti, che lavorano insieme su una raccolta di oggetti condivisi.

4.6 ULTERIORI LETTURE

Una leggibile introduzione al tema della gestione della configurazione è data in (Babich, **1986**). Una fonte più recente è (Jonassen Hass, **2002**). (Estublier *et al.*, **2005**) offre un'eccellente panoramica dei principali sviluppi nel campo. Weber (**1996**) e Wiborg-Weber (**1997**) descrivono la tecnologia di gestione della configurazione orientata al cambiamento.

Ulteriori riferimenti per aspetti tecnici della gestione della configurazione sono forniti nel **Capitolo 15** .

Esercizi

1. Quali sono i compiti principali della gestione della configurazione?

2. Descrivere il ruolo di un comitato di controllo di configurazione.

3. Che cos'è un elemento di configurazione?

4. Che cos'è una “baseline” ?

5. Spiegare la differenza tra la gestione della configurazione orientata alla versione e quella orientata al cambiamento.

6. Discutere i contenuti principali di un piano di gestione della configurazione.

7. ♡ Discutere differenze e somiglianze tra la gestione della configurazione durante le fasi di sviluppo e manutenzione di un progetto.

8. ♡ Discutere le possibili differenze tra la gestione della configurazione in un modello di sviluppo tradizionale a cascata e in un modello di sviluppo evolutivo (vedi anche (Bersoff e Davis, **1991**)).

9. ♡ La gestione della configurazione a livello di implementazione è spesso supportata da strumenti. Riuscite a pensare ai modi in cui tali strumenti possono anche supportare il controllo di altri artefatti (come documenti di progettazione e rapporti di prova)?

10. ♠ Elaborare uno schema di gestione della configurazione per un piccolo progetto (diciamo, meno di un anno-persona) e un grande progetto (diciamo, più di dieci anni-persona). Fornire una motivazione per le possibili differenze tra questi schemi.

11. ♠ In che misura gli strumenti di gestione della configurazione possono supportare la raccolta di dati quantitativi di progetto? In che misura tali strumenti potrebbero supportare il controllo del progetto?